

## Regression Algorithm – Insurance Charges Prediction

### ❖ Problem Statement:

The client wants to predict the insurance charges of a person based on several parameters such as age, sex, BMI, number of children, and smoking status. The provided dataset contains **1338 rows and 6 columns**.

### ❖ Pre-processing:

The dataset contains categorical/string values such as **sex** and **smoker**. I used **One-Hot Encoding** to convert the categorical values into numerical values so that they can be used by machine learning algorithms.

### ❖ Algorithms Used:

- Random Forest Regressor
- Decision Tree Regressor
- Multiple Linear Regression
- Support Vector Machine
- Ada Boost Regressor

### ❖ Evaluation Metric:

I used the  **$R^2$  (R-squared) score** to compare the performance of the regression models.

- **LGBoosting Regression:**

```
] from lightgbm import LGBMRegressor
regressor = LGBMRegressor(n_estimators=100, learning_rate=0.1, max_depth=-1, random_state=42)
regressor.fit(X_train, y_train)
```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.00036s. You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 313  
[LightGBM] [Info] Number of data points in the train set: 936, number of used features: 5  
[LightGBM] [Info] Start training from score 13232.916456

```
] LGBMRegressor ⓘ
  ▶ Parameters
  ▶ Fitted attributes
```

```
] y_pred=regressor.predict(X_test)
from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

```
] 0.86603193419773
```

```
import pickle
filename="finalized_lg_boost.sav"
```

```
pickle.dump(regressor,open(filename,'wb'))
```

```
loaded_model=pickle.load(open("finalized_lg_boost.sav",'rb'))
```

```
result=loaded_model.predict([[19,27.90,0,0,1]])
```

```
C:\anaconda3\Lib\site-packages\sklearn\utils\validation.py:282: FutureWarning:
The `r2_score` function is deprecated in favor of the `r2` function.
warnings.warn(
```

```
result
```

```
array([17506.89176097])
```

## ➤ Ada Boost Regressor:

```
from sklearn.ensemble import AdaBoostRegressor
regressor=AdaBoostRegressor(random_state=0, n_estimators=100)
regressor.fit(X_train,y_train)
```

C:\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1365: se change the shape of y to (n\_samples, ), for example using `rav`  
y = column\_or\_1d(y, warn=True)

▼ AdaBoostRegressor ⓘ ?

- ▶ Parameters
- ▶ Fitted attributes

```
y_pred=regressor.predict(X_test)
from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

0.8447476929154872

```
26]: y_pred=regressor.predict(X_test)
      from sklearn.metrics import r2_score
      r_score=r2_score(y_test,y_pred)
      r_score
```

26]: 0.8447476929154872

```
27]: import pickle
      filename="finalized_ada_boost.sav"
```

```
28]: pickle.dump(regressor,open(filename,'wb'))
```

```
29]: loaded_model=pickle.load(open("finalized_ada_boost.sav",'rb'))
```

```
30]: result=loaded_model.predict([[19,27.90,0,0,1]])
```

C:\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2827: User  
h feature names  
warnings.warn(

```
33]: result
```

```
33]: array([19235.18177276])
```

## ➤ XGBoost Regression:

```
from xgboost import XGBRegressor
regressor = XGBRegressor(n_estimators=100, learning_rate=0.05, max_depth=3, random_state=42)
regressor.fit(X_train, y_train)
```

▼ XGBRegressor ⓘ ?

- ▶ Parameters
- ▶ Fitted attributes

```
y_pred=regressor.predict(X_test)
from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

0.8922968506813049

```
import pickle
filename="finalized_XG_boost_regressor.sav"
pickle.dump(regressor,open(filename,'wb'))
loaded_model=pickle.load(open("finalized_XG_boost_regressor.sav",'rb'))
result=loaded_model.predict([[19,27.90,0,0,1]])
result
```

array([18180.2], dtype=float32)

## ➤ Support Vector Machine:

### Without standardization:

```
: from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

```
: -0.12207668380229886
```

### With Standardization:

```
[25]: from sklearn.metrics import r2_score
      r_score=r2_score(y_test,y_pred)
      r_score
```

```
[25]: -0.080959968427891
```

## ➤ Random Forest:

```
: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(independent,dependent,test_size=0.3,random_state=0)

: from sklearn.ensemble import RandomForestRegressor
regressor=RandomForestRegressor(n_estimators=100,random_state=0)
regressor=regressor.fit(X_train,y_train)

C:\anaconda3\Lib\site-packages\sklearn\base.py:1403: DataConversionWarning: A column-vector y was passed as a
e shape of y to (n_samples,), for example using ravel().
    return fit_method(estimator, *args, **kwargs)

: y_pred=regressor.predict(X_test)

from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score

: 0.8539781318598829
```

## ➤ Decision Tree:

```
: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(independent,dependent,test_size=0.3,random_state=0)

: from sklearn.ensemble import RandomForestRegressor
regressor=RandomForestRegressor(n_estimators=100,random_state=0)
regressor=regressor.fit(X_train,y_train)

C:\anaconda3\Lib\site-packages\sklearn\base.py:1403: DataConversionWarning: A column-vector y was passed as a
e shape of y to (n_samples,), for example using ravel().
    return fit_method(estimator, *args, **kwargs)

: y_pred=regressor.predict(X_test)

from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score

: 0.8539781318598829
```

## ➤ Multi Linear Regression:

```
: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(independent,dependent,test_size=0.30,random_state=0)

: from sklearn.linear_model import LinearRegression
regressor=LinearRegression()
regressor.fit(X_train,y_train)

: ▼ LinearRegression ⓘ ?
  ▶ Parameters
  ▶ Fitted attributes

: y_pred=regressor.predict(X_test)
  from sklearn.metrics import r2_score
  r_score=r2_score(y_test,y_pred)
  r_score

: 0.7894790349867009
```

### ❖ Best Model:

The **Random Forest Regressor** performed the best, with an **R<sup>2</sup> score of 0.853**.

### ❖ Conclusion:

Among the three regression algorithms, the **Random Forest Regressor** gave the highest R<sup>2</sup> score of **0.853**. Therefore, Random Forest is the best-performing model for predicting insurance charges in this dataset. An R<sup>2</sup> score of 0.853 means that the model explains approximately **85.3% of the variation in insurance charges** based on the input feature